Knowledge Handover Platform

An evidence-linked, uncertainty-aware continuation plan for people who take over someone else's work

1. Our Understanding of the Problem Statement

When someone leaves a project (resignation, rotation, graduation, end of a volunteer term), the work does not stop but the context does. What the leaver knows lives in many places at once: pull requests, Slack threads, Jira tickets, Google Docs and meeting summaries. These sources disagree with each other, omit the reasoning behind decisions, and use informal language.

The problem statement asks for more than a document export. A good handover must reconstruct the real state of the project, show how sure the system is about each statement, let a responsible human correct it, check it automatically, and then share it with the incoming person **only at the level their permissions allow**. The core tension is trust: an AI guess presented as verified truth, or a confidential file leaked to the wrong person, makes the tool worse than having no tool at all.

2. The Specific Problem We Are Solving

Incoming owners cannot tell what is true, what is uncertain, and what is missing. They inherit stale wikis and rushed exit meetings, and spend weeks rebuilding context. Existing handover tools are wikis or exporters: they dump data and hope someone reads it.

We are solving this narrower, concrete problem: *given the sources a leaver had access to, produce a reviewed, validated, role-scoped continuation plan in which every claim links to its evidence, carries a confidence score, and is explicitly marked unknown or conflicting when evidence is missing or contradictory.*

3. Who Experiences the Problem and Who Our Users Are

User	How they experience the problem	What they get from our product
Incoming person (new owner, new hire)	Spends weeks reconstructing ownership, risks and next steps; cannot verify stale documentation.	A role-scoped continuation plan on day one, with clickable citations and only the content they are authorised to see.
Departing person (leaver)	Faces a rushed exit and a blank Google Doc; worries that knowledge is lost or misattributed.	Their knowledge is captured with attribution and a structured, reviewable plan.
Reviewer (manager, tech lead)	Has no reliable way to confirm that a handover is complete, correct and safe to share.	A review UI to accept, reject or edit each claim, plus an automated validation gate and an approval trail.
Team / organisation	Operational disruption, hidden single points of failure, no compliance record.	Shorter transitions, surfaced dependencies, and an immutable record of who approved what and when.

Primary target teams: engineering teams, student clubs and volunteer groups with departing members.

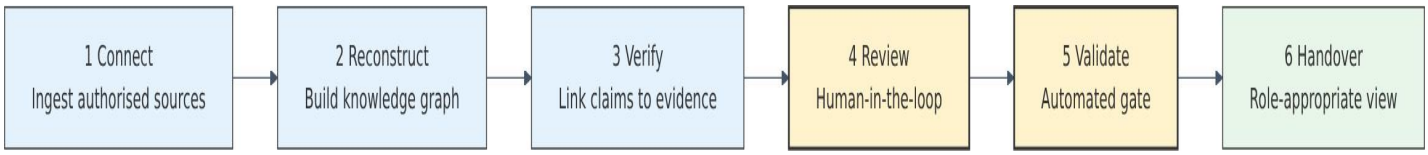
4. Our Proposed Solution

A web platform that connects to the leaver's authorised sources, reconstructs the project as a **knowledge graph**, and delivers a continuation plan. Our bet has three parts:

- **Evidence linking, not generation.** Every claim carries a citation to a specific source fragment (for example PR #412, a Slack message, Jira PROJ-88). The system never invents an answer; missing evidence is shown as a gap.
- **Uncertainty as a first-class field.** Claims start as *proposed*, carry a 0.0-1.0 confidence score, and are flagged when two sources conflict (for example a doc says "API v2 is deprecated" while a ticket says "shipping v2 next week").
- **Permissions that flow end to end.** Source ACLs propagate to claims and are enforced at
- render time, so the incoming person sees only what they are authorised to access, not merely what existed in a connected source.

The output is a **continuation plan** (owners, blockers, dependencies, risks, next actions) rather than a summary document.

Figure 1. The six-layer handover pipeline.



5. The Complete Product Workflow

The workflow has six layers. Two of them (Review and Validate) are deliberate trust checkpoints: a human approves the content, then an automated gate re-checks it before anything is shared.

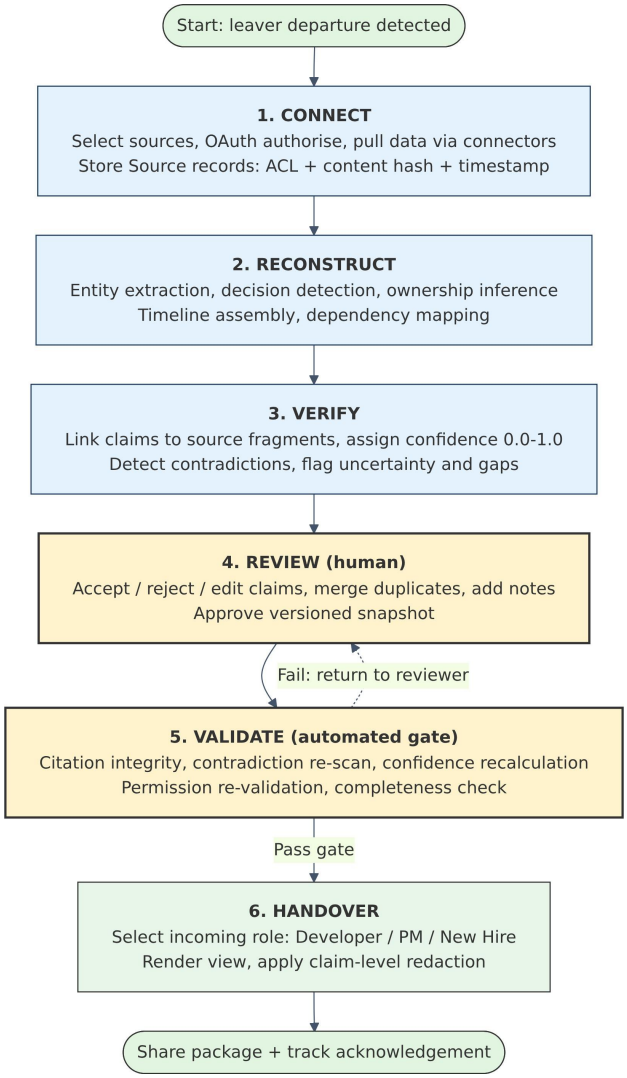


Figure 2. End-to-end workflow, from leaver departure to acknowledged handover.

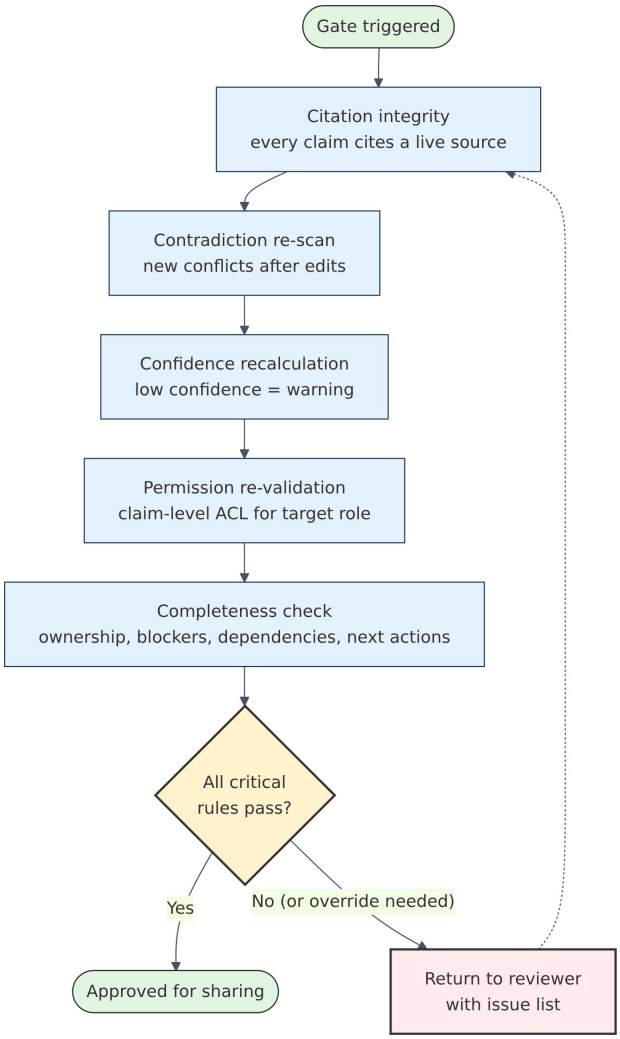


Figure 3. Validation gate. Critical failures block publication until fixed or explicitly overridden by an approver.

Workflow in words

Layer	What happens	Key output
1. Connect	Authorised connectors pull from sources the leaver could access. Every item is stored append-only with its original ACL, content hash and timestamp.	Source records
2. Reconstruct	Entity extraction, decision detection, ownership inference (CODEOWNERS, assignees), timeline assembly, dependency mapping.	Typed nodes: Decision, Task, Question, Dependency, Risk, Contact
Layer	**What happens**	**Key output**
3. Verify	Claims linked to source fragments; confidence from source authority (code > ticket > doc > chat) and corroboration; contradictions and gaps flagged.	Cited, scored claims
4. Review	A responsible person accepts, rejects or edits each claim, merges duplicates, adds notes and approves a versioned snapshot. Every action is audit-logged.	Approved snapshot
5. Validate	Citation integrity, contradiction re-scan, confidence recalculation, permission re-validation, completeness check. Pass/fail gate.	Validation report
6. Handover	Role view (Developer / PM / New Hire) rendered with claim-level redaction; recipient acknowledges receipt.	Shared continuation plan + acknowledgement

Sequence: ingesting a source

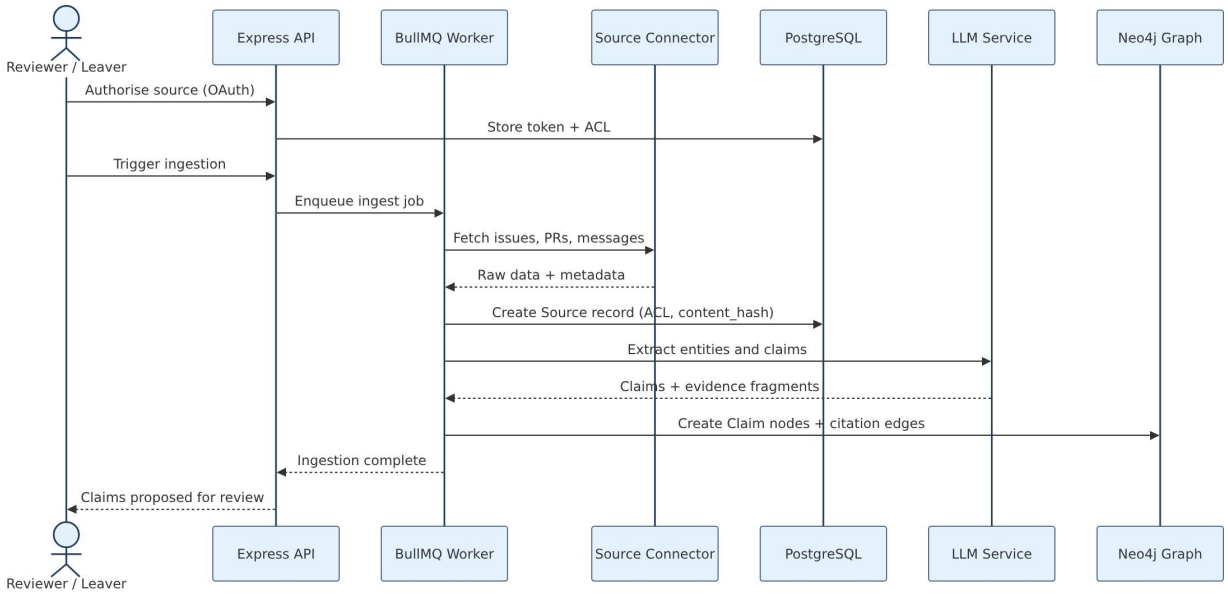


Figure 4. Authorise, ingest, extract, and write claims with citation edges.

Sequence: review and approval

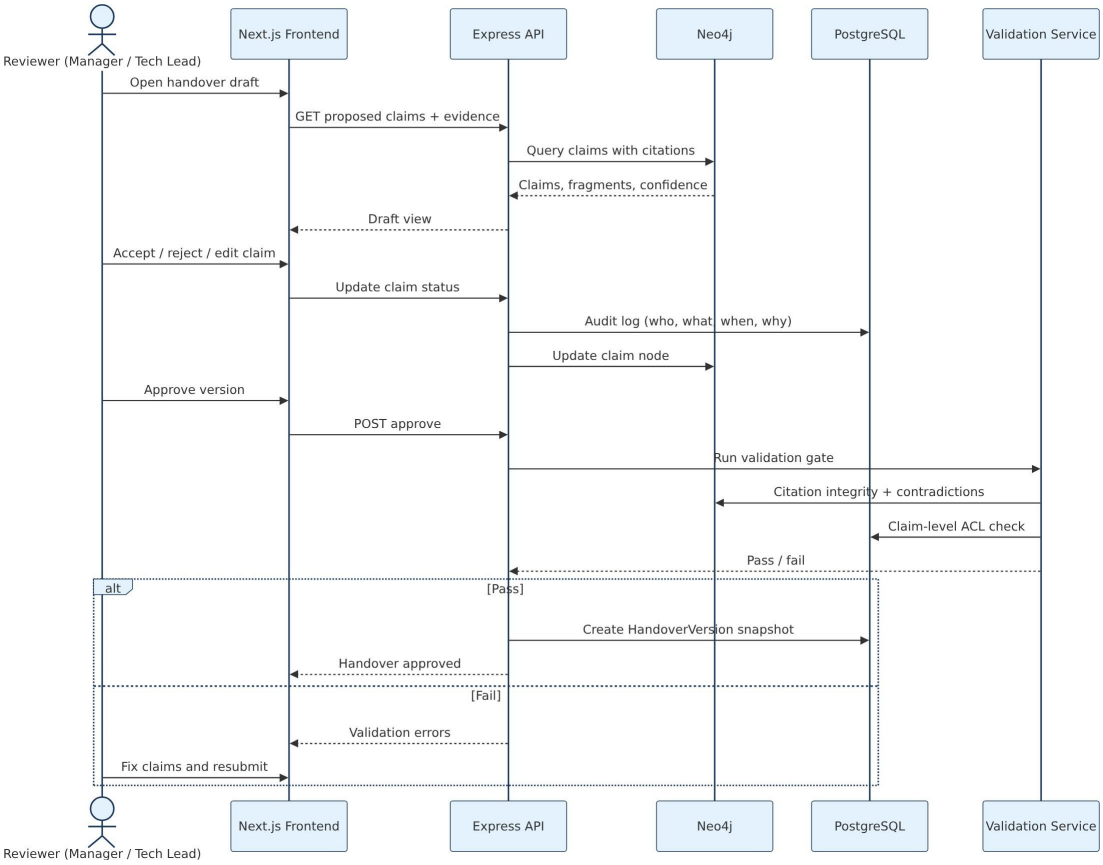


Figure 5. Reviewer edits claims; approval triggers the validation gate.

Sequence: role-scoped handover rendering

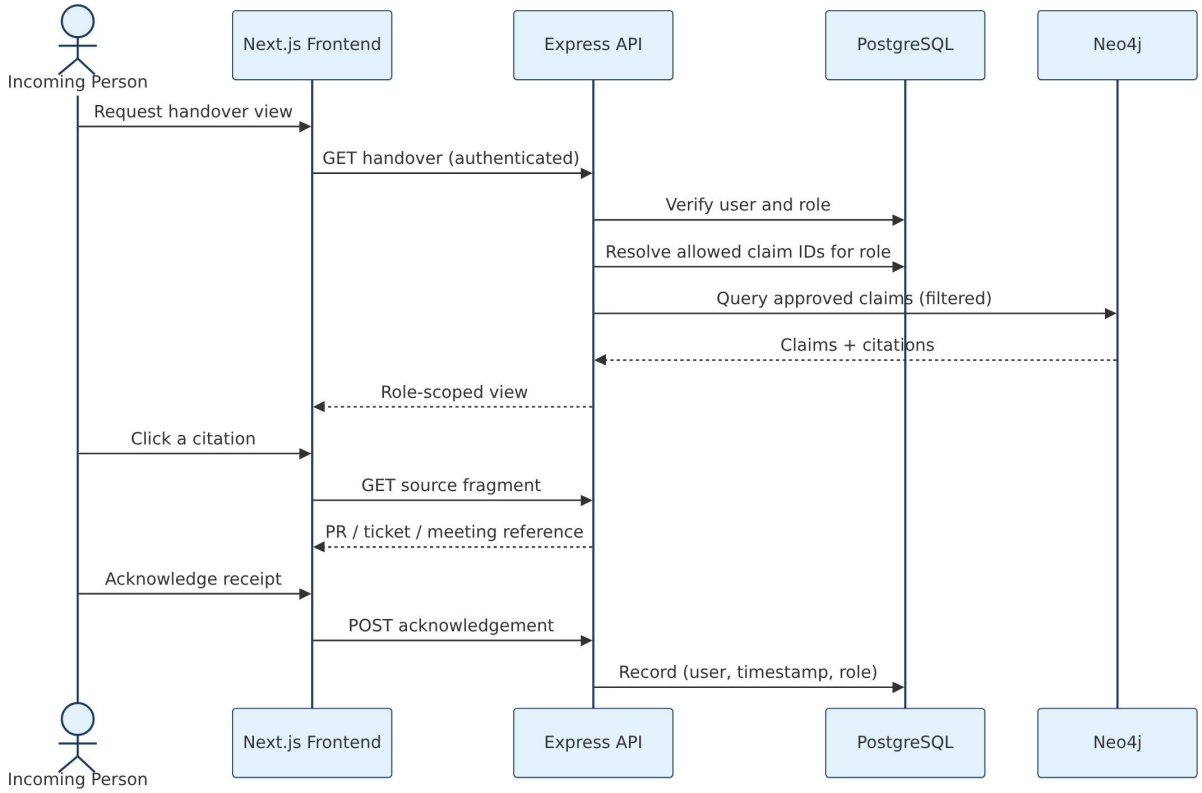


Figure 6. The incoming person sees only authorised claims, can click through to evidence, and acknowledges receipt.

6. Main Features We Intend to Build

#	Feature	Description	Round
1	GitHub connector	Fetches issues, PRs, comments and CODEOWNERS; normalises to a common Source schema with ACL and content hash.	2
2	Knowledge-graph reconstruction	LLM-assisted decision, entity, ownership and dependency extraction stored in Neo4j with citation edges.	2
3	Evidence links and confidence	Each claim cites source fragments; scored 0.0-1.0; claims below 0.7 auto-flag as uncertain; missing evidence shown as gaps.	2
4	Contradiction detection	Flags conflicting claims across sources for human decision instead of choosing a side.	2
5	Claim review UI	Accept / reject / edit claims with citations and confidence visible; audit log of who, what, when, why.	2
6	Automated validation gate	Citation integrity, contradiction re-scan, confidence recalculation, permission re-validation, completeness check; blocks publication on critical failure.	2
7	Role-scoped handover views	Developer, PM and New Hire views with claim-level redaction and clickable citations; acknowledgement tracking.	2
8	Google Drive connector	Docs, Sheets and Slides with inherited ACLs.	3

#	Feature	Description	Round
9	AI meeting-summary connector	Zoom AI Companion, Google Meet summary, Fireflies.ai, Otter.ai. Treated as lower-authority sources (confidence capped around 0.6 until corroborated).	3
10	On-demand code version comparison	User picks two versions (for example v5 vs v3); system surfaces architecturally significant changes linked to the authorising PR or ticket.	3
11	Synthetic demo dataset	Fake repo and tickets, clearly labelled synthetic, so judges can test without real data.	2-3

Phase 3: on-demand code version comparison

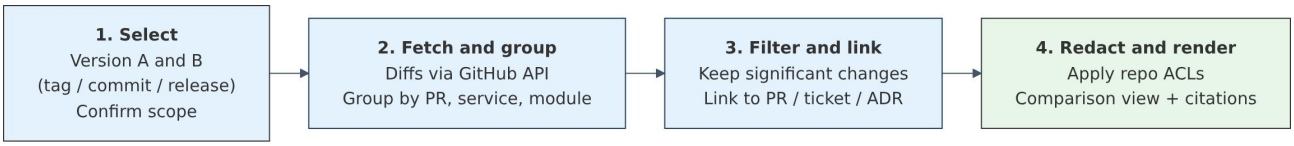


Figure 7. The comparison is never automatic: it runs only when a user selects the two versions.

7. Technology Stack and Architecture

Layer	Technology	Cost	Notes
Runtime	Node.js 18+ LTS	Free	Download from nodejs.org
Backend API	Express.js	Free	Simple, easy to learn, huge ecosystem
Graph database	Neo4j Community Edition	Free	Self-hosted desktop app on localhost:7474
Relational DB	PostgreSQL + pgvector	Free	Supabase free tier or local install
Cache / queue	Redis	Free	brew install redis, or Redis Cloud free tier
Object storage	Local /uploads folder	Free	Development; switch to S3 later if needed
Frontend	Next.js + TypeScript + Tailwind CSS	Free	Node.js-based, familiar ecosystem
Auth	Clerk or Passport.js + JWT	Free tier	Clerk has a generous free tier; Passport is fully free
AI / LLM	Hugging Face Inference API (Llama-3.1-8B-Instruct) or Ollama (local)	Free	HF: ~30 req/min limit; Ollama: unlimited, local
LLM library	node-fetch (HF) or openai SDK (Ollama-compatible)	Free	
Background jobs	BullMQ + Redis	Free	Ingestion workers
CI/CD	GitHub Actions	Free	For public repos

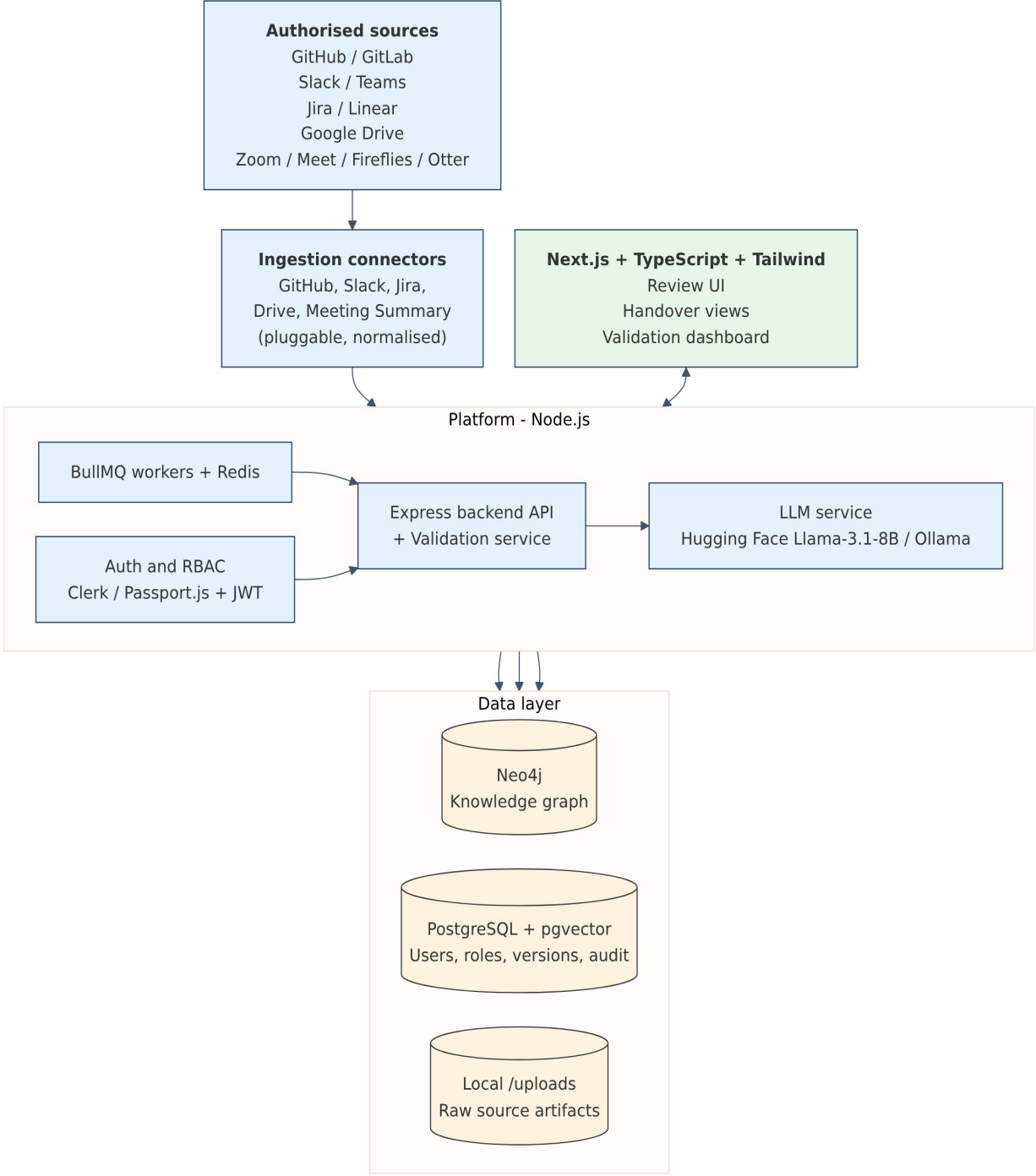


Figure 8. System architecture.

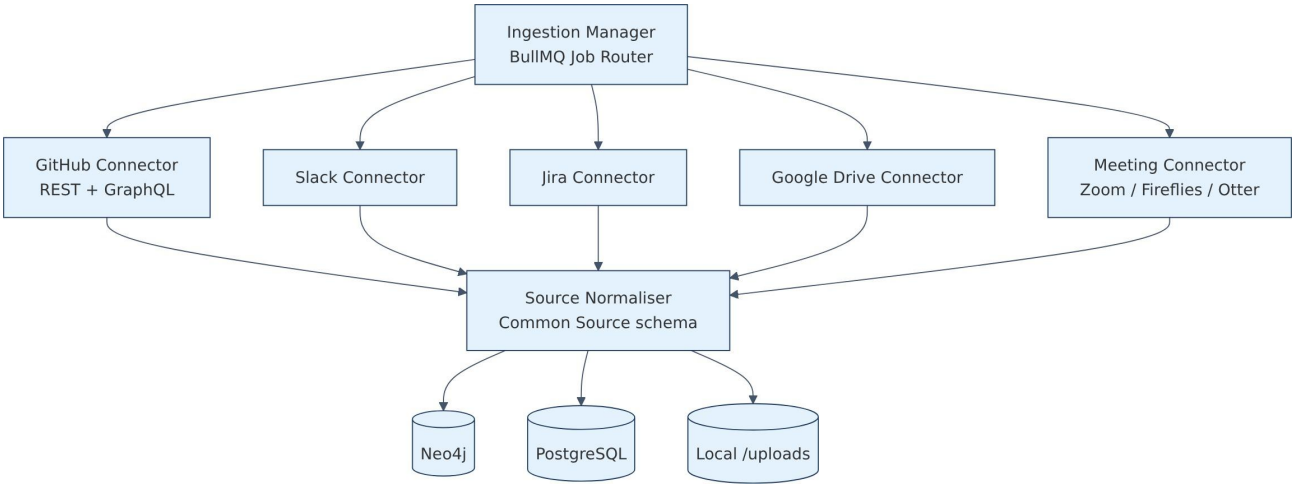


Figure 9. Pluggable connector architecture: every connector normalises into the same Source schema.

Knowledge graph schema

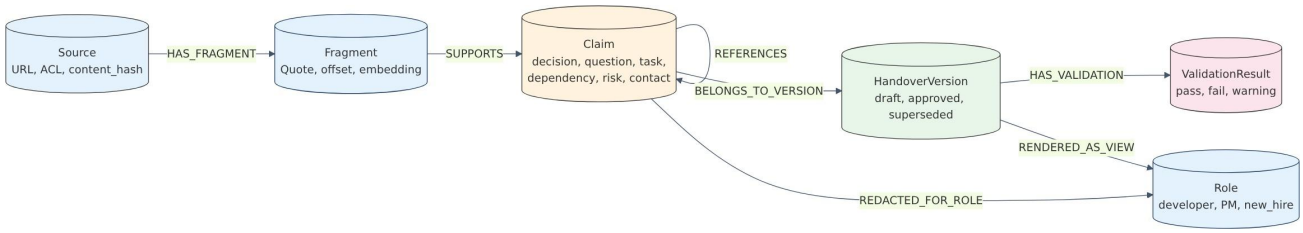


Figure 10. Neo4j node and relationship types. Relational data (users, roles, versions, audit, validation results) lives in PostgreSQL.

Core data model

Entity	Key fields
Source	id, type, url, content_hash, acl, ingested_at, raw_content
Fragment	id, source_id, quote, start_offset, end_offset, embedding
Claim	id, type (decision | question | task | dependency | risk | contact), text, confidence, status (proposed | accepted | rejected | edited)
ClaimEvidence / Contradiction	claim_id, fragment_id, weight / claim_a_id, claim_b_id, description
HandoverVersion / HandoverView	id, project_id, leaver_id, approver_id, status / version_id, role_id, redacted_node_ids, rendered_content
Acknowledgement	handover_id, recipient_id, acknowledged_at, role_at_handover
ValidationResult / ValidationRule	rule_id, status (pass | fail | warning), detail / type, severity (critical | warning), enabled

8. Why We Selected This Technology Stack

Choice	Reason
Node.js + Express	The team is most comfortable with JavaScript/TypeScript, which cuts onboarding time and bugs in a time-boxed hackathon. Express is minimal with no hidden magic, and it shares a language ecosystem (types, utilities) with the Next.js frontend.

Choice	Reason
Neo4j Community	The core innovation is an evidence-linked graph. A relational table cannot express "this decision contradicts that ticket" or "this task depends on that PR" as naturally. Neo4j Desktop installs without Docker.
PostgreSQL + pgvector	Users, roles, versions, audit logs and validation results are relational. pgvector stores fragment embeddings for semantic search of conflicting claims, without a separate vector database.
Hugging Face Llama-3.1-8B + Ollama	The LLM is used only for extraction, contradiction detection, confidence scoring and summarisation, not chat. HF has a free tier with strong structured-JSON reasoning; Ollama runs locally with no rate limits so development is never blocked. One environment variable (LLM_PROVIDER) switches providers.
Next.js + TypeScript + Tailwind	TypeScript catches interface errors early, which matters when the view must render claims, citations and redactions correctly. Tailwind lets us spend time on functionality rather than CSS.
BullMQ + Redis	Ingestion is slow and rate-limited; background jobs keep the API responsive and stay in the Node.js ecosystem.
No Docker; all free tiers	Docker setup would consume feature time. All services install directly on macOS/Windows. Every component is free, so the project is reproducible by any judge or future contributor.

9. Major Technical Challenge and How We Will Handle It

Challenge: reconstructing a reliable project state from scattered, unstructured and contradictory sources. If the system presents an LLM guess as verified truth, it fails the trust requirement. An 8B-parameter model will also make mistakes, so the design must assume it is fallible.

Mitigation	How it works
1. Evidence linking	Every claim has a citation to a specific fragment; the incoming person can click through to verify.
2. Uncertainty first-class	Claims default to status proposed. Confidence reflects source authority (code > ticket > doc > chat) and corroboration count. Missing evidence is a visible gap.
3. Contradiction detection	Conflicting statements about the same fact are flagged for human review; the system never picks a side.
4. Human-in-the-loop	A manager or tech lead accepts, rejects or edits every claim before publication.
5. Automated validation gate	After human edits, citation integrity, contradictions, permissions and completeness are re-checked. Critical failures block publication.
6. Constrained LLM use	The model must return structured JSON that quotes a fragment; claims without a valid quote are dropped, not published.

Secondary risks: Hugging Face free-tier limits (~30 requests/min) are handled by queueing in BullMQ with retry and by falling back to Ollama; permission leaks are handled by propagating source ACLs to claims and re-validating at the gate; scope risk is handled by shipping a single connector end to end in Round 2.

10. Team Responsibilities

Member	Role	Primary focus
Member 1	Backend Engineer	Express API, PostgreSQL, Neo4j, authentication
Member 2	Frontend Engineer	Next.js UI, React components, handover views, citations
Member 3	AI & Data Engineer	Hugging Face / Ollama LLM, source connectors, graph reconstruction
Member 4	Integration & QA Engineer	Validation gate, BullMQ workers, testing, deployment, documentation

What each member personally builds

Member	Deliverables personally owned
Member 1 Backend	Database schema (PostgreSQL + Neo4j); auth strategy and implementation; Express routes (POST /ingest, GET /sources, /auth/*); Source and Fragment storage with ACL and content hash; audit-log storage; Round 2 bug fixes (auth edge cases, Neo4j connection drops, query optimisation); backend README and API docs.
Member 2 Frontend	Wireframes for the ingestion dashboard, claim review UI and handover view; Next.js project and layout; claim review UI with confidence and uncertainty display; role-scoped Developer / PM / New Hire views with claim-level redaction and clickable citations; UI polish (loading, empty and error states, responsiveness); frontend README.
Member 3 AI / Data	LLM service (llm.js) with Hugging Face and Ollama and its test; prompt templates for decision extraction and contradiction detection; GitHub connector; claim extraction, entity recognition and confidence scoring; Neo4j claim graph with citations; Google Drive and meeting-summary connectors; LLM usage documentation.
Member 4 Integration / QA	Research of connector APIs, rate limits and OAuth; Redis and BullMQ workers; automated validation gate (five checks, pass/fail); deployment to a public URL; on-demand code version comparison; end-to-end tests including permission-aware redaction; GitHub repo setup and branch protection; user guide and final production deploy.

11. Development Plan Across All Three Rounds

Round 1 - The Plan (2 days): case, architecture and setup, no production code

Day	Member 1	Member 2	Member 3	Member 4
1	Define DB schema; choose auth; sketch API routes	Wireframe 3 core screens; choose UI library	Compare HF vs Ollama; test rate limits; draft prompts	Research connector APIs and OAuth; draft no-Docker deployment plan
2	Init backend repo; set up PostgreSQL and Neo4j; .env.example	Init Next.js + TS + Tailwind; layout and routing	Build llm.js (HF + Ollama) and llm.test.js	Set up Redis; GitHub branch protection; README

Deliverable: plan PDF, repo initialised, local environments running, LLM test passing.

Round 2 - The Build (3 days): end-to-end working product on a public URL

Day	Work	Owners
1 Backend and ingestion	Express routes; Source records in PostgreSQL; Neo4j Source/Fragment nodes; GitHub connector (issues, PRs, comments, CODEOWNERS); ingestion worker pushes to graph; verify GitHub to Express to PostgreSQL + Neo4j.	Members 1 and 3
2 Reconstruction, verification, review	LLM decision extraction, entity recognition, confidence scoring and contradiction detection; claims in Neo4j with citations; claim review UI; connect frontend to backend; audit log.	Members 3 and 2
Day	**Work**	**Owners**
3 Validation gate and handover	Validation gate (5 checks); role-scoped views with redaction and clickable citations; deploy to Render / Railway / Vercel; verify the full flow including acknowledgement.	Members 4 and 2

Deliverable: a live product: connect a GitHub repo, review generated claims, pass the validation gate and receive a role-appropriate handover.

Round 3 - The Finish (3 days): polish into a product people would use

Day	Work	Owners
1 Feature completion	Google Drive connector with inherited ACLs; meeting-summary connector; on-demand code version comparison; fix Round 2 bugs from judge feedback.	Members 3, 4 and 1
2 Polish and testing	UI/UX tightening (loading, empty, error states, citation rendering, mobile); end-to-end tests of every main journey; permission-redaction test cases; bug bash.	Members 2 and 4
3 Documentation and final deploy	Backend and frontend READMEs; LLM usage notes; reviewer and incoming-person user guide; synthetic end-to-end test; final production deploy.	All members

Deliverable: polished, documented, deployed product with Drive and meeting connectors, code comparison, validation gate and role-scoped views.

Is the scope realistic?

Round 2 deliberately limits ingestion to **one** connector (GitHub) so that the full chain (ingest, graph, review, validate, handover) works end to end before breadth is added. Slack, Jira, Linear and Markdown docs are designed for (the connector interface is pluggable) but are stretch goals, not commitments.

Rounds 1 and 3 are staffed so that no member is idle and no round depends on a single person finishing late.

12. Expected Impact of the Solution

For	Impact
Incoming person	Understands ownership, risks and next steps on day one instead of spending weeks; verifies decisions through citations rather than trusting stale docs; sees only what they are authorised to access.
Departing person	Leaves a structured continuation plan instead of a blank doc and a rushed exit meeting; knowledge is preserved with attribution, reducing pressure and guilt.
Team	Shorter transitions and less disruption; an immutable approval trail (who approved what, when, which version) for compliance; previously invisible dependencies and recurring problems become visible.

13. Future Scope of the Product

- **Near term:** Slack, Jira / Linear and Markdown connectors; Confluence, Notion, Microsoft Teams and Azure Boards; real-time source sync instead of batch ingestion.
- **Code comparison:** richer semantic diffs between any two releases, always linked to authorising PRs, tickets and ADRs.
- **Privacy:** fully self-hosted deployments using local LLMs (Ollama) for organisations with strict data-privacy needs.
- **Analytics:** knowledge-gap dashboard (services with a single owner, decisions with no documentation).
- **Reach:** mobile app for reviewing handovers; HR offboarding integrations (BambooHR, Workday) that trigger a handover automatically when a departure is recorded.

14. Complete Team and Participant Details

Please fill in the bracketed fields before submitting.

Member	Full name	Email / phone	College / organisation	Role	Part they will build
Member 1 (Team lead)	Basabee Hati	Hatibb20@gmail.com	SR University , Warangal	Backend Engineer	API, PostgreSQL, Neo4j, auth
Member 2	Akshaya Gouraram	akshayagouraram@gmail.com	SR University , Warangal	Frontend Engineer	Review UI, handover views, citations
Member 3	Rimjhim Rani	rimjhim580960@gmail.com	SR University , Warangal	AI & Data Engineer	LLM service, connectors, graph reconstruction
Member 4	Ragni Kumari	ragnikumariedu2@gmail.com	SR University , Warangal	Integration & QA Engineer	Validation gate, workers, tests, deploy